Sequential Task:

Cafe *Java*

Object-Oriented Programming and GUI

March 9, 2026

1. Declare the record `Product`, which will represent a single product in the cafe offer and store the basic data describing that product.

   - Define three record components: `name`, `price`, and `category`.
   - Treat the record as an immutable data structure describing a product.
   - Use the fact that a record automatically generates a constructor, accessor methods, `equals`, `hashCode`, and `toString`.

   (a) Add a compact constructor to the `Product` record in order to validate the data passed during object creation.

   - Check the correctness of the parameters before saving them in the record.
   - Ensure that every created object satisfies the logical requirements of the application.

   i. Verify the correctness of the product name passed in the `name` parameter to ensure that a product always has a valid name.

   - Check whether the value is not equal to `null`.
   - Check whether the text is not empty and does not consist only of whitespace.
   - If the condition is not satisfied, throw an `IllegalArgumentException` with an appropriate message.

   Example exception message

   Product name cannot be empty.

   ii. Verify the product price passed in the `price` parameter to ensure the correctness of financial data.

   - Check whether the price is greater than zero.
   - If the value is invalid, throw an `IllegalArgumentException`.
   - Include the received price value in the error message to make diagnostics easier.

   Example exception message

   Price must be greater than 0. Received: -5.0

iii. Verify the correctness of the product category passed in the `category` parameter so that every product is assigned to a specific group.
   - Check whether the value is not equal to `null`.
   - Check whether the category text is not empty and does not consist only of whitespace.
   - In case of an error, throw an `IllegalArgumentException` with an appropriate message.

(b) Add the method `formatted`, whose task will be to return a `String` that clearly describes the product for presentation to the user, e.g.:
"Cappuccino (coffee) - 12.50 PLN"
   - use the static `format` method from the `String` class;
   - let the `format` method take four arguments: a format string describing the syntax and then the values of the fields `name`, `category`, `price`

2. Declare the record `Customer`, which represents a cafe customer participating in the loyalty program and stores the basic identifying data of the customer.

(a) Define three record components: `name`, `email`, and `loyaltyPoints`.

(b) Treat the record as an immutable data structure storing customer information.

(c) Add a compact constructor to the `Customer` record whose task is to validate the data passed during object creation.

i. Verify that the `name` parameter is not equal to `null` and is not an empty or blank string using the `isBlank()` method.

ii. If the condition is not satisfied, throw an `IllegalArgumentException` with a message stating that the customer name cannot be empty.

iii. Verify that the `email` parameter is not equal to `null` and contains the `"@"` character, which allows the address to be considered basically valid.

iv. If the email address does not meet the validity condition, throw an `IllegalArgumentException` containing a message stating that the email address is invalid and providing the received value.

v. Verify that the `loyaltyPoints` parameter is not less than zero.

vi. If the number of loyalty points is negative, throw an `IllegalArgumentException` with a message stating that the number of points cannot be negative.

(d) Implement the method `loyaltyLevel()`, whose task is to determine the customer's loyalty level based on the number of collected points.
   - If the number of points `loyaltyPoints` is greater than or equal to `200`, return the level `"GOLD"`.
   - If the number of points is greater than or equal to `100`, return the level `"SILVER"`.
   - If the number of points is greater than or equal to `50`, return the level `"BRONZE"`.

- Otherwise, return the level `"STANDARD"`.

(e) Implement the method `formatted()`, whose task is to prepare a readable textual representation of the customer.

- Use the String.format method to create a string containing the customer name, loyalty level, and number of points.
- In the formatting content, use the values of the `name` and `loyaltyPoints` components.
- Call the `loyaltyLevel()` method to insert the customer's current loyalty level into the created string.
- Return the formatted text as the method result.

3. Present your work progress to the instructor.

4. Declare the abstract class `Discount`, which will represent a general discount mechanism and enforce the implementation of price calculation after discount in subclasses.

(a) Add a private final immutable field `description` of type `String`, which stores the textual description of the discount displayed on the receipt.

(b) Implement the constructor `Discount(String description)`, whose task is to initialize the discount description and validate the passed data.

i. Verify that the passed `description` parameter is not equal to `null` and is not an empty or blank string using the `isBlank()` method.

ii. If the validity condition is not satisfied, throw an `IllegalArgumentException` with a message stating that the discount description cannot be empty.

iii. If validation succeeds, assign the passed description to the `description` field.

(c) Add a public accessor method `getDescription()`, whose task is to return the value of the `description` field.

(d) Declare the abstract method `apply`, which determines how to calculate the price after applying a discount.

- The method takes the parameter `double originalPrice`, representing the price before discount.
- The parameter value should be greater than or equal to zero.
- The method returns the price after applying the discount.
- The result of the method should not be less than zero.
- The method implementation is to be provided in classes inheriting from `Discount`.

(e) Implement the method `savings(double originalPrice)`, whose task is to calculate the discount amount, that is, the difference between the original price and the discounted price.

- Call the `apply(originalPrice)` method to obtain the discounted price.
- Subtract the received value from the original price.
- Return the calculated difference as the savings value.

(f) Override the `toString()` method so that it returns a readable textual description of the discount.

- Return a string consisting of the prefix `"Discount:  "` and the value of the `description` field.

5. Declare the class `FixedAmountDiscount`, which represents a fixed discount amount subtracted from the total order price, and make it inherit from the abstract class `Discount`.

   (a) Add a private immutable field `amount`, which will store the discount value expressed as a fixed amount subtracted from the order price.

   (b) Implement the constructor `FixedAmountDiscount(double amount)`, whose task is to set the discount description and validate the passed discount value.

      i. Call the constructor of the parent class `Discount`, passing a discount description created using String.format, which formats the `amount` value into text containing information about the discount amount.

      ii. Verify that the passed `amount` value is greater than zero.

      iii. If the discount value is less than or equal to zero, throw an `IllegalArgumentException` containing a message stating that the discount value is invalid together with the received number.

      iv. If validation succeeds, assign the `amount` parameter value to the `amount` field.

   (c) Add a public accessor method `getAmount()`, whose task is to return the value of the `amount` field.

   (d) Override the method `apply(double originalPrice)` to define how to calculate the price after applying a fixed-amount discount.

      - Subtract the value of the `amount` field from the passed price `originalPrice`.
      - Use the `Math.max` method to make sure the result is not less than zero.
      - Return the calculated discounted price.

6. Declare the class `PercentageDiscount`, which represents a percentage discount applied to the order price, and make it inherit from the abstract class `Discount`.

   (a) Add a private immutable field `percentage`, which will store the discount value expressed as a percentage.

   (b) Implement the constructor `PercentageDiscount(double percentage)`, whose task is to set the discount description and validate the discount percentage value.

      i. Call the constructor of the parent class `Discount`, passing a discount description created using String.format, which formats the `percentage` value as text containing information about the percentage discount.

      ii. Verify that the value of the `percentage` parameter is greater than zero and does not exceed `100`.

iii. If the percentage value is outside the range `(0, 100]`, throw an `IllegalArgumentException` with a message stating that the discount percentage must be within the valid range and providing the received value.

> Example exception message
>
> Discount percentage must be in the range (0, 100]. Received: 150.

iv. If validation succeeds, assign the `percentage` parameter value to the `percentage` field.

(c) Add a public accessor method `getPercentage()`, whose task is to return the value of the `percentage` field.

(d) Override the method `apply(double originalPrice)` to define how to calculate the price after applying a percentage discount.

- Calculate the discounted price by multiplying `originalPrice` by the expression `(1 - percentage / 100.0)`.
- Store the result in a helper variable named `discounted`.
- Use the `Math.max` method to make sure the resulting price is not less than zero.
- Return the calculated discounted price.

7. Present your work progress to the instructor.

8. Declare the record `OrderItem`, which represents a single order line consisting of a product and the number of ordered units.

(a) Define two record components: `product` and `quantity`.

(b) Treat the record as an immutable data structure describing one order line.

(c) Add a compact constructor to the `OrderItem` record, whose task is to validate the data passed during object creation.

i. Verify that the `product` parameter is not equal to `null`.

ii. If the passed product is equal to `null`, throw an `IllegalArgumentException` with a message stating that the product cannot be `null`.

> Example exception message
>
> product cannot be empty

iii. Verify that the `quantity` parameter is greater than or equal to `1`.

iv. If the quantity is less than `1`, throw an `IllegalArgumentException` containing a message stating that the quantity must be at least `1` and providing the received value.

> Example exception message
>
> Quantity must be at least 1. Received: 0

(d) Implement the method `totalPrice()`, whose task is to calculate the total price for a given order line.

- Retrieve the product unit price using the accessor method `price()` of the `Product` record.
- Multiply the unit price by the value of `quantity`.
- Return the calculated value as the total price of the order line.

(e) Implement the method `formatted()`, whose task is to prepare a formatted textual description of the order line.

- Create the left part of the line containing the product quantity and name using String.format.
- Create the right part of the line containing the formatted line price with two decimal places and the currency unit.
- Define the target width of the whole line as a constant numeric value, e.g. 40.
- Calculate the number of spaces needed to align the text between the left and right parts.
- Use the method `" ".repeat(...)` to insert the required number of spaces.
- Join the left part of the line, the generated spaces, and the right part into one string.
- Return the result as the formatted description of the order line.

9. Declare the class `Order`, which will represent an order in the cafe, store its identifier, customer, items saved in an array, creation moment, and an optional discount.

(a) Add a private field `id` of type `int`, which stores the order identifier.

(b) Add a private final immutable field `items` of type `OrderItem[]`, which stores order lines in an array without using collections.

(c) Add a private final immutable field `customer` of type `Customer`, which stores the data of the customer placing the order.

(d) Add a private final immutable field `createdAt` of type `LocalDateTime`, which stores the creation time of the order.

(e) Add a private field `discount` of type `Discount`, which stores the currently assigned discount and may be changed after object creation.

(f) Implement the private constructor `Order(int id, OrderItem[] items, Customer customer, Discount discount)`, whose task is to initialize the full state of the order.

i. Assign the passed `id` parameter to the `id` field.

ii. Create a new `OrderItem[]` array with a length equal to the length of the passed `items` array in order to perform a defensive copy.

iii. Copy the elements from the input array to the newly created array using a `for` loop, without using `Arrays.copyOf`.

iv. Assign the copied array to the `items` field.

v. Assign the passed `customer` object to the `customer` field.

vi. Assign the passed `discount` object to the `discount` field.

vii. Set the `createdAt` field to the current date and time using `LocalDateTime.now()`.

(g) Add the method `getId()`, which returns the value of the `id` field.

(h) Add the method `getCustomer()`, which returns the object stored in the `customer` field.

(i) Add the method `getDiscount()`, which returns the current discount object stored in the `discount` field.

(j) Add the method `setDiscount(Discount discount)`, which allows replacing the discount assigned to the order by saving the passed argument in the `discount` field.

(k) Implement the method `getItems()`, whose task is to return a defensive copy of the order items array.

i. Create a new `OrderItem[]` array with a length equal to the length of the `items` field.

ii. Copy all elements from the `items` field to the new array using a `for` loop.

iii. Return the created copy instead of a direct reference to the internal field.

(l) Implement the method `getLineCount()`, which returns the number of order lines stored in the order by reading the length of the `items` array.

(m) Implement the method `getItemCount()`, whose task is to calculate the total number of product units across all order lines.

i. Initialize an accumulator variable `count` with the value `0`.

ii. Iterate through all elements of the `items` array using a `for` loop.

iii. For each order line, retrieve the quantity by calling `quantity()` and add it to the `count` variable.

iv. Return the final value of the `count` variable.

(n) Implement the method `calculateSubtotal()`, whose task is to calculate the sum of all order line values before applying the discount.

i. Initialize the variable `sum` with the value `0`.

ii. Iterate through all elements of the `items` array.

iii. For each order line, call `totalPrice()` and add the returned value to the `sum` variable.

iv. Return the calculated sum as the subtotal value.

(o) Implement the method `calculateTotal()`, whose task is to calculate the final amount to pay after considering an optional discount.

i. Call `calculateSubtotal()` and store the result in a helper variable `subtotal`.

ii. Check whether the `discount` field is not equal to `null`.

iii. If the discount exists, call `discount.apply(subtotal)` and return the result.

iv. If no discount has been set, return the value of `subtotal`.

(p) Override the `toString()` method so that it returns a short order description.

- Use String.format to build a string containing the order number, customer name, total number of units, and final order value.
- Retrieve the customer name by calling `customer.name()`.
- Retrieve the number of units by calling `getItemCount()`.
- Retrieve the final amount by calling `calculateTotal()`.

10. Add a public non-static inner class `Receipt`, which will generate a textual representation of a receipt with access to the private fields of the enclosing `Order` instance.

(a) Add a private final immutable field `cashierName`, which stores the name of the cashier handling the receipt.

(b) Add a private constant `CAFE_NAME`, which stores the cafe name displayed in the receipt header. For simplicity, assume it will be `"CAFE "UNDER JAVA""`.

(c) Add a private constant `WIDTH`, which defines the target width of the textual receipt printout, e.g. 42.

(d) Implement the constructor `Receipt(String cashierName)`, whose task is to validate and save the cashier name.

i. Verify that the `cashierName` parameter is not equal to `null` and is not an empty or blank string.

ii. If the validity condition is not satisfied, throw an `IllegalArgumentException` with a message stating that the cashier name cannot be empty.

iii. If validation succeeds, assign the passed value to the `cashierName` field.

(e) Implement the private method `center(String text)`, whose task is to center the passed text within the receipt width.

- Calculate the number of leading spaces as half of the difference between `WIDTH` and the text length, not allowing a negative value.
- Return a string consisting of the proper number of spaces and the passed text.

(f) Implement the private method `formatLine(String label, double amount)`, whose task is to format a normal summary line with alignment.

- Build the left part of the line as indentation plus the label.
- Build the right part of the line as the amount with two decimal places and the `PLN` suffix.
- Calculate the number of spaces needed to align both parts to the width `WIDTH`, ensuring at least one space.
- Join the left part, generated spaces, and right part, then return the result.

(g) Implement the private method `formatNegLine(String label, double amount)`, whose task is to format an aligned line presenting a negative value, for example a discount.

- Build the left part of the line analogously to the `formatLine` method.
- Build the right part of the line as an amount preceded by a minus sign.
- Calculate the number of spaces needed to align the text to the width `WIDTH`.
- Join all parts and return the finished line.

(h) Implement the method `generate()`, whose task is to build the complete receipt text.

i. Create a `StringBuilder` object that will accumulate the receipt lines.

ii. Prepare two helper separator strings by repeating the `"="` and `"-"` characters to the width defined by `WIDTH`.

iii. Add the top separator, the centered cafe name, and the next separator to the result.

iv. Create a DateTimeFormatter object with the pattern `dd.MM.yyyy HH:mm`.

v. Add a line with the date, formatting the `createdAt` field using the prepared formatter.

vi. Add a line with the cashier name.

vii. Add a line with the order number using the `id` field.

viii. Add a line with the customer name and loyalty level using `customer.name()` and `customer.loyaltyLevel()`.

ix. Add an empty line separating the header from the items.

x. Iterate through the private `items` array using a `for` loop and for each order line append the result of the `formatted()` method followed by a newline character.

xi. Add an empty line and a thin separator separating the items from the summary.

xii. Calculate the subtotal value by calling `calculateSubtotal()`.

xiii. Add the line with the pre-discount total by calling the helper method `formatLine("Subtotal:", subtotal)`.

xiv. Check whether the `discount` field is not equal to `null`.

A. If the discount exists, calculate the savings by calling `discount.savings(subtotal)`.

B. Add the line with the discount description and deducted amount by calling `formatNegLine("Discount:  " + discount.getDescription(), savings)`.

xv. Add a thin separator before the final section.

xvi. Add the line with the final amount due by calling `formatLine("TOTAL DUE:", calculateTotal())`.

xvii. Add the bottom separator, a centered thank-you message, and the last separator.

xviii. Return the result of `sb.toString()`.

11. Add a public static inner class `Builder`, which will be responsible for step-by-step construction of an `Order` object and manual management of a dynamically growing items array.

(a) Add a private constant `INITIAL_CAPACITY`, which defines the initial size of the items array, e.g. 4.

(b) Add a private final immutable field `id`, which stores the identifier of the order being built.

(c) Add a private final immutable field `customer`, which stores the customer assigned to the order being built.

(d) Add a private field `items` of type `OrderItem[]`, which stores the internal array of items expanded during building.

(e) Add a private field `size`, which stores the current number of actually added elements.

(f) Add a private field `discount`, which stores an optional discount set during the building stage.

(g) Implement the constructor `Builder(int id, Customer customer)`, whose task is to validate the input data and prepare an empty structure for building the order.

i. Verify that the `id` parameter is greater than zero.

ii. If the identifier does not satisfy the condition, throw an `IllegalArgumentException` with a message stating that the order ID must be greater than zero.

iii. Verify that the `customer` parameter is not equal to `null`.

iv. If the customer has not been provided, throw an `IllegalArgumentException` with a message stating that the customer cannot be equal to `null`.

v. Assign the valid identifier to the `id` field.

vi. Assign the valid customer object to the `customer` field.

vii. Create an `OrderItem[]` array of size `INITIAL_CAPACITY` and assign it to the `items` field.

viii. Set the `size` field to the value `0`.

(h) Implement the private method `grow()`, whose task is to double the capacity of the items array when there is no space left.

i. Create a new `OrderItem[]` array whose length is twice the length of the current `items` array.

ii. Copy all elements from index `0` to `size - 1` into it.

iii. Replace the reference in the `items` field with the newly created larger array.

(i) Implement the method `addItem(Product product, int quantity)`, whose task is to add a new item to the order being built.

i. Check whether the current number of stored elements `size` is equal to the length of the `items` array.

ii. If the array is full, call the `grow()` method.

iii. Create a new `OrderItem` object based on the passed product and quantity.

iv. Save the new item at index `size`, then increase the value of `size` by one.

v. Return the current `Builder` object to enable method chaining.

(j) Implement the overloaded method `addItem(Product product)`, which adds a product in the default quantity of one unit.

- Call the method `addItem(product, 1)`.
- Return the result of that call.

(k) Implement the method `withDiscount(Discount discount)`, which assigns a discount to the order being built.

- Save the passed discount in the `discount` field.
- Return the current `Builder` object to keep the fluent building style.

(l) Implement the method `build()`, whose task is to create a complete `Order` object based on the data collected in the builder.

i. Verify that the number of added items `size` is greater than zero.

ii. If the order contains no items, throw an `IllegalStateException` with a message stating that the order must contain at least one item.

iii. Create a new `OrderItem[]` array whose length is equal to the current value of `size` in order to trim the extra capacity.

iv. Copy all actually added elements from the working `items` array into it.

v. Create and return a new `Order` object, passing `id`, the trimmed items array, `customer`, and `discount`.

12. Present your work progress to the instructor.

13. Declare the class `Cafe`, which will manage the cafe name, the product menu stored in an array, and the orders stored in a separate array, and perform all collection-like operations manually without using collections.

(a) Add a private final immutable field `name`, which stores the cafe name.

(b) Add a private field `menu` of type `Product[]`, which stores the products available in the menu.

(c) Add a private field `menuSize`, which stores the number of actually saved products in the `menu` array.

(d) Add a private field `orders` of type `Order[]`, which stores registered orders.

(e) Add a private field `orderCount`, which stores the number of actually saved orders in the `orders` array.

(f) Implement the constructor `Cafe(String name, int menuCapacity, int orderCapacity)`, whose task is to validate the input parameters and prepare the initial working arrays.

i. Verify that the `name` parameter is not equal to `null` and is not an empty or blank string.

ii. If the name does not satisfy the validity condition, throw an `IllegalArgumentException` with a message stating that the cafe name cannot be empty.

iii. Verify that the `menuCapacity` and `orderCapacity` parameters are greater than zero.

iv. If either capacity does not satisfy the condition, throw an `IllegalArgumentException` with a message stating that the capacities must be greater than zero.

v. Assign the valid name to the `name` field.

vi. Create a `Product[]` array of size `menuCapacity` and assign it to the `menu` field.

vii. Set the `menuSize` field to the value `0`.

viii. Create an `Order[]` array of size `orderCapacity` and assign it to the `orders` field.

ix. Set the `orderCount` field to the value `0`.

(g) Implement the method `addProduct(Product product)`, whose task is to add a new product to the menu and automatically expand the array if needed.

i. Verify that the `product` parameter is not equal to `null`.

ii. If the product has not been provided, throw an `IllegalArgumentException` with a message stating that the product cannot be equal to `null`.

iii. Check whether the number of stored elements `menuSize` is equal to the length of the `menu` array.

iv. If the array is full, create a new `Product[]` array whose size is equal to twice the length of the current array.

v. Copy all existing products from the old array into the new array using a `for` loop.

vi. Replace the reference in the `menu` field with the new larger array.

vii. Save the passed product at index `menuSize`, then increase the value of `menuSize` by one.

(h) Implement the method `removeProduct(String productName)`, whose task is to remove a product from the menu based on its name and shift the remaining elements in the array.

i. Initialize the variable `index` with the value `-1` to mark that no matching element has been found.

ii. Iterate through all saved products from index `0` to `menuSize - 1`.

iii. Compare the name of each product with the `productName` parameter using `equalsIgnoreCase()`.

iv. If you find a matching product, save its index in the `index` variable and stop further searching using `break`.

v. Check whether after the search the value of `index` is still `-1`.

vi. If the product was not found, return `false`.

vii. If the product was found, shift all following array elements one position to the left, starting from the found index up to the last occupied element before the end.

viii. Decrease the value of `menuSize` by one, and set the freed last slot to `null`.

ix. Return `true` to confirm successful removal of the product.

(i) Implement the method `getProductsByCategory(String category)`, whose task is to filter products belonging to the specified category and return a new result array.

i. Initialize the counter `count` with the value `0`.

ii. Iterate through all saved products in the menu and for each one compare the value of `category()` with the passed parameter using `equalsIgnoreCase()`.

iii. For each matching product, increase the value of the `count` counter.

iv. Create a new `Product[]` array whose length is equal to the number of found matches.

v. Initialize a helper index `idx` with the value `0`.

vi. Iterate through all saved products again.

vii. For each product belonging to the specified category, place it into the result array at index `idx`, then increase `idx` by one.

viii. Return the filled result array.

(j) Implement the method `sortMenuByPrice()`, whose task is to sort the products in ascending order by price using the `Insertion Sort` algorithm.

i. Start iterating from index `1` to `menuSize - 1`.

ii. Save the current element in the helper variable `key`.

iii. Set the helper index `j` to the position directly preceding the current element.

iv. As long as the index `j` does not go past the beginning of the array and the price of `menu[j]` is greater than the price of `key`, shift `menu[j]` one position to the right and decrease the value of `j`.

v. After shifting is complete, insert the `key` element at position `j + 1`.

(k) Implement the method `displayMenu()`, whose task is to print the whole menu in a readable text form.

- Print a header containing the cafe name written in uppercase letters by calling `name.toUpperCase()`.
- Iterate through all saved products from index `0` to `menuSize - 1`.
- For each product, print the item number and the result of the `formatted()` method.
- At the end, print an empty line separating the output from the following text.

(l) Implement the method `addOrder(Order order)`, whose task is to add a new order to the register and automatically expand the array when there is no more space.

i. Verify that the `order` parameter is not equal to `null`.

ii. If the order has not been provided, throw an `IllegalArgumentException` with a message stating that the order cannot be equal to `null`.

iii. Check whether the value of `orderCount` is equal to the length of the `orders` array.

iv. If the array is full, create a new `Order[]` array whose size is equal to twice the length of the current array.

v. Copy all existing orders into the new array using a `for` loop.

vi. Replace the reference in the `orders` field with the new larger array.

vii. Save the passed order at index `orderCount`, then increase the value of `orderCount` by one.

(m) Implement the method `getOrdersByCustomer(String customerName)`, whose task is to filter orders belonging to the specified customer and return them in a new array.

i. Initialize the counter `count` with the value `0`.

ii. Iterate through all saved orders from index `0` to `orderCount - 1`.

iii. For each order, retrieve the customer name by calling `getCustomer().name()` and compare it with the `customerName` parameter using `equalsIgnoreCase()`.

iv. For each matching order, increase the value of the `count` counter.

v. Create a new `Order[]` array whose length is equal to the number of found matches.

vi. Initialize a helper index `idx` with the value `0`.

vii. Iterate through all saved orders again.

viii. For each matching order, place it into the result array at index `idx`, then increase `idx` by one.

ix. Return the filled result array.

(n) Implement the method `sortOrdersByTotal()`, whose task is to sort orders in ascending order by final value using the `Bubble Sort` algorithm.

i. Run an outer loop controlling the number of passes from the beginning to the last but one comparable element.

ii. In each iteration, run an inner loop comparing adjacent elements from the beginning of the array to the last unsorted position.

iii. For each pair of adjacent orders, compare the results of `calculateTotal()`.

iv. If the current order has a greater value than the next one, swap the two elements using a temporary variable.

(o) Add the method `getName()`, which returns the value of the `name` field.

(p) Add the method `getMenuSize()`, which returns the value of the `menuSize` field.

(q) Add the method `getOrderCount()`, which returns the value of the `orderCount` field.

(r) Implement the method `getOrders()`, whose task is to return a trimmed copy of the orders array.

i. Create a new `Order[]` array whose length is equal to the value of `orderCount`.

ii. Copy all actually saved orders from the `orders` array into it.

iii. Return the created array as a defensive copy.

14. Add a public static inner class `Statistics`, which will perform statistical calculations on the passed array of orders without the need to refer to a `Cafe` instance.

(a) Add a private final immutable field `orders`, which stores the array of analyzed orders.

(b) Add a private final immutable field `count`, which stores the number of analyzed elements in the array.

(c) Implement the constructor `Statistics(Order[] orders, int count)`, whose task is to validate the input data and save it in the object fields.

i. Verify that the `orders` parameter is not equal to `null`.

ii. Verify that the `count` parameter is greater than zero.

iii. If either condition is not satisfied, throw an `IllegalArgumentException` with a message stating that there are no orders to analyze.

iv. Assign the passed array to the `orders` field.

v. Assign the passed number of elements to the `count` field.

(d) Implement the method `totalRevenue()`, whose task is to calculate the total revenue from all analyzed orders.

i. Initialize the variable `sum` with the value `0`.

ii. Iterate through the array elements from index `0` to `count - 1`.

iii. For each order, retrieve the final value by calling `calculateTotal()` and add it to the `sum` variable.

iv. Return the calculated sum.

(e) Implement the method `averageOrderValue()`, whose task is to calculate the average order value.

- Call the `totalRevenue()` method.
- Divide the obtained result by the value of the `count` field.
- Return the calculated average.

(f) Implement the method `mostExpensiveOrder()`, whose task is to find the order with the highest value.

i. Assume the first element of the array is the current maximum and save it in the variable `max`.

ii. Iterate through the following elements of the array from index `1` to `count - 1`.

iii. For each order, compare the result of `calculateTotal()` with the value of the current maximum.

iv. If the current order has a greater value, assign it to the variable `max`.

v. Return the found order.

(g) Implement the method `cheapestOrder()`, whose task is to find the order with the lowest value.

i. Assume the first element of the array is the current minimum and save it in the variable `min`.

ii. Iterate through the following elements of the array from index `1` to `count - 1`.

iii. For each order, compare the result of `calculateTotal()` with the value of the current minimum.

iv. If the current order has a smaller value, assign it to the variable `min`.

v. Return the found order.

(h) Implement the method `totalItemsSold()`, whose task is to calculate the total number of sold units across all analyzed orders.

i. Initialize the variable `total` with the value `0`.

ii. Iterate through the array elements from index `0` to `count - 1`.

iii. For each order, call the method `getItemCount()` and add the returned value to the `total` variable.

iv. Return the calculated sum.

(i) Implement the method `summary()`, whose task is to prepare a textual summary of the statistics.

- Create a `StringBuilder` object and append the statistics section header to it.
- Add lines with the number of orders, the number of sold units, the total revenue, and the average order value, using String.format.
- Add a line with the identifier and value of the most expensive order by calling `mostExpensiveOrder().getId()` and `mostExpensiveOrder().calculateTotal()`.
- Add a line with the identifier and value of the cheapest order by calling `cheapestOrder().getId()` and `cheapestOrder().calculateTotal()`.
- Return the result of `sb.toString()`.

15. Add a public non-static inner class `DailyReport`, which will generate a daily report with direct access to the private fields of the enclosing `Cafe` instance.

(a) Add a private final immutable field `reportDate`, which stores the report date as text.

(b) Implement the constructor `DailyReport(String reportDate)`, whose task is to validate the passed report date.

i. Verify that the `reportDate` parameter is not equal to `null` and is not an empty or blank string.

ii. If the report date does not satisfy the validity condition, throw an `IllegalArgumentException` with a message stating that the report date cannot be empty.

iii. If validation succeeds, assign the passed value to the `reportDate` field.

(c) Implement the method `generate()`, whose task is to build the complete daily report.

i. Create a `StringBuilder` object that will accumulate the report lines.

ii. Prepare a separator string by repeating the `"="` character fifty times.

iii. Add to the result the top separator, the line with the report name and cafe name, and the line with the report date.

iv. Add another separator and an empty line.

v. Insert information about the number of menu products and the number of orders, directly using the `menuSize` and `orderCount` fields.

vi. Add an empty line separating the header section from the detailed data.

vii. Check whether the value of `orderCount` is greater than zero.

A. If orders exist, add the order list header.

B. Iterate through all orders from index `0` to `orderCount - 1`.

C. For each order, append a formatted line containing the order identifier, customer name, total value, and number of units, using String.format, `getId()`, `getCustomer().name()`, `calculateTotal()`, and `getItemCount()`.

D. After the order list is finished, add an empty line.

E. Create a `Statistics` object, passing the `orders` array and the `orderCount` value.

F. Add to the report the line with total revenue by calling `stats.totalRevenue()`.

G. Add to the report the line with average order value by calling `stats.averageOrderValue()`.

viii. If there are no orders, append to the report a message stating that no orders are available.

ix. Add an empty line, the bottom separator, and a newline character.

x. Return the result of `sb.toString()`.

16. Test the program using the attached `Demo.java` class, compare the result, and present it to the instructor. The result of running the program in the console should be:

```
=== MENU: UNDER JAVA ===

  1. Cappuccino (coffee) - 12.50 PLN
  2. Espresso (coffee) - 7.50 PLN
  3. Latte (coffee) - 14.00 PLN
  4. Americano (coffee) - 9.00 PLN
  5. Cheesecake (cake) - 14.50 PLN
  6. Brownie (cake) - 11.00 PLN
  7. Croissant (pastry) - 8.50 PLN
  8. Earl Grey Tea (tea) - 9.00 PLN

(after sorting by price)

=== MENU: UNDER JAVA ===

  1. Espresso (coffee) - 7.50 PLN
  2. Croissant (pastry) - 8.50 PLN
  3. Americano (coffee) - 9.00 PLN
  4. Earl Grey Tea (tea) - 9.00 PLN
  5. Brownie (cake) - 11.00 PLN
  6. Cappuccino (coffee) - 12.50 PLN
  7. Latte (coffee) - 14.00 PLN
  8. Cheesecake (cake) - 14.50 PLN

=== COFFEES ON OFFER ===
```

```
  Espresso (coffee) - 7.50 PLN
  Americano (coffee) - 9.00 PLN
  Cappuccino (coffee) - 12.50 PLN
  Latte (coffee) - 14.00 PLN

=== CUSTOMERS ===

  Anna [SILVER] (150 pts)
  Tomek [STANDARD] (30 pts)
  Kasia [GOLD] (250 pts)

=== ORDER 1 ===

Number of lines: 3
Number of units: 6
Subtotal:        62.00 PLN
Total:           55.80 PLN

==========================================
           CAFE "UNDER JAVA"
==========================================
Date:       06.03.2026 22:07
Cashier:    Anna
Order:      #1
Customer:   Anna [SILVER]

  2x Cappuccino                 25.00 PLN
  1x Cheesecake                 14.50 PLN
  3x Espresso                   22.50 PLN

------------------------------------------
  Subtotal:                       62.00 PLN
  Discount: 10.0% off             -6.20 PLN
------------------------------------------
  TOTAL DUE:                      55.80 PLN
==========================================
               Thank you!
==========================================

=== ORDER 2 ===

==========================================
```

```
           CAFE "UNDER JAVA"
========================================
Date:       06.03.2026 22:07
Cashier:    Tomek
Order:      #2
Customer:   Tomek [STANDARD]

  1x Latte                       14.00 PLN
  2x Croissant                   17.00 PLN

----------------------------------------
  Subtotal:                       31.00 PLN
  Discount: 5.00 PLN off         -5.00 PLN
----------------------------------------
  TOTAL DUE:                      26.00 PLN
========================================
               Thank you!
========================================

=== ORDER 3 (loyalty discount - anonymous class) ===

========================================
           CAFE "UNDER JAVA"
========================================
Date:       06.03.2026 22:07
Cashier:    Kasia
Order:      #3
Customer:   Kasia [GOLD]

  2x Earl Grey Tea               18.00 PLN
  1x Cheesecake                  14.50 PLN
  2x Brownie                     22.00 PLN

----------------------------------------
  Subtotal:                       54.50 PLN
  Discount: GOLD Loyalty         -13.63 PLN
----------------------------------------
  TOTAL DUE:                      40.88 PLN
========================================
               Thank you!
========================================
```

```
=== ORDER 4 (no discount) ===

========================================
           CAFE "UNDER JAVA"
========================================
Date:       06.03.2026 22:07
Cashier:    Anna
Order:      #4
Customer:   Anna [SILVER]

  3x Americano                  27.00 PLN
  1x Croissant                   8.50 PLN

----------------------------------------
  Subtotal:                      35.50 PLN
----------------------------------------
  TOTAL DUE:                     35.50 PLN
========================================
               Thank you!
========================================

=== ORDER RANKING (ascending by amount) ===

  Order #2 (Tomek) ->  26.00 PLN  (3 units)
  Order #4 (Anna) ->  35.50 PLN  (4 units)
  Order #3 (Kasia) ->  40.88 PLN  (5 units)
  Order #1 (Anna) ->  55.80 PLN  (6 units)

=== RANKING BY NUMBER OF UNITS (descending) ===

  Order #1 -> 6 units (Anna)
  Order #3 -> 5 units (Kasia)
  Order #4 -> 4 units (Anna)
  Order #2 -> 3 units (Tomek)

=== ANNA'S ORDERS ===

  #4 -> 35.50 PLN
  #1 -> 55.80 PLN
  Total: 91.30 PLN (2 orders)

=== STATISTICS ===
```

```
  Number of orders:   4
  Units sold:         18
  Total revenue:      158.18 PLN
  Average value:      39.54 PLN
  Most expensive:     #1 (55.80 PLN)
  Cheapest:           #2 (26.00 PLN)

==================================================
  DAILY REPORT: Under Java
  Date: 2025-01-15
==================================================

  Products in menu: 8
  Orders:           4

  --- Order list ---
    #2 | Tomek      |  26.00 PLN | 3 units
    #4 | Anna       |  35.50 PLN | 4 units
    #3 | Kasia      |  40.88 PLN | 5 units
    #1 | Anna       |  55.80 PLN | 6 units

  Total revenue:    158.18 PLN
  Average value:     39.54 PLN

==================================================

=== VALIDATION TESTS ===

  [OK] Product: Product name cannot be empty.
  [OK] Product: Price must be greater than 0. Received: -5.0
  [OK] OrderItem: Quantity must be at least 1. Received: 0
  [OK] PercentageDiscount: Discount percentage must be in the range (0, 100]. Received:
  [OK] FixedAmountDiscount: Discount amount must be > 0. Received: -10.0
  [OK] Customer: Invalid email address: no-at-sign
  [OK] Builder: Order must contain at least one item.

=== CHANGING DISCOUNT ON THE FLY ===

  Order #4 BEFORE change:  35.50 PLN
  Order #4 AFTER 15% discount: 30.18 PLN
```

```
========================================
           CAFE "UNDER JAVA"
========================================
Date:       06.03.2026 22:07
Cashier:    Manager
Order:      #4
Customer:   Anna [SILVER]

  3x Americano                   27.00 PLN
  1x Croissant                    8.50 PLN

----------------------------------------
  Subtotal:                       35.50 PLN
  Discount: 15.0% off             -5.32 PLN
----------------------------------------
  TOTAL DUE:                      30.18 PLN
========================================
               Thank you!
========================================

=== REMOVING A PRODUCT FROM THE MENU ===

  Before: 8 products
  Removed 'Brownie': true
  After:  7 products

=== MENU: UNDER JAVA ===

  1. Espresso (coffee) - 7.50 PLN
  2. Croissant (pastry) - 8.50 PLN
  3. Americano (coffee) - 9.00 PLN
  4. Earl Grey Tea (tea) - 9.00 PLN
  5. Cappuccino (coffee) - 12.50 PLN
  6. Latte (coffee) - 14.00 PLN
  7. Cheesecake (cake) - 14.50 PLN
```